

CS 442 | Mobile Application Development with Flutter & Dart

WEEK 1

Mobile Landscape

Flutter Architecture & Dev Setup

Part A — 45 min
Native vs Cross-Platform

Part B — 45 min
Dev Setup + First App

BS-CS | Semester 7 | Fall 2026

1

Today's Session — Roadmap

PART A — 45 MIN

The Mobile Landscape

1. The Android & iOS duopoly
2. Cross-platform history: PhoneGap → Flutter
3. How Flutter renders differently
4. When NOT to use Flutter
5. Q&A + mental model check

PART B — 45 MIN

Dev Setup & First Flutter App

1. Install Flutter SDK + VS Code
2. Configure Android emulator (AVD)
3. Run flutter doctor — fix all issues
4. Run the default counter app
5. Enhance it: Reset + threshold message

2

The Mobile Duopoly — Two Worlds, One Problem

Android

- Language: Kotlin (formerly Java)
- IDE: Android Studio
- UI: Jetpack Compose / XML layouts
- Market share: ~72% globally
- Distribution: Google Play Store
- Build output: AAB / APK

iOS

- Language: Swift (formerly Obj-C)
- IDE: Xcode (macOS only)
- UI: SwiftUI / UIKit
- Market share: ~27% globally
- Distribution: Apple App Store
- Build output: IPA / TestFlight

Two languages · Two IDEs · Two skill sets · Two release pipelines — very costly for small teams.

3

Cross-Platform: A Brief History of Compromises

2008

PhoneGap / Cordova

HTML + CSS + JS in a WebView. Web apps pretending to be mobile apps.

Slow. Didn't look native.

2011

Xamarin

C# shared codebase compiled to native widgets on each platform.

Complex. Expensive.

2015

React Native

JavaScript + React mapped to real native UI elements.

JS bridge overhead.

2018

Flutter

Dart language. Renders every pixel itself — no bridge, no native widgets.

Solves the bridge problem.

4

Flutter's Answer — It Draws Everything Itself

React Native

JS → Bridge → Native Widgets

- Separate thread for JS
- Bridge = performance cost
- Native widgets look different per OS

Flutter

Dart → Skia/Impeller → Pixels

- No bridge whatsoever
- Own rendering engine
- Pixel-perfect on ALL platforms

Native (Kotlin/Swift)

Platform code → Platform Widgets

- Best performance possible
- Platform-specific only
- Two separate codebases needed

One codebase · **Two app stores** — The same Dart code compiles to a real Android APK and a real iOS IPA.

5

Flutter Architecture — The 4-Layer Stack

Your Flutter App (Dart)

Widgets, business logic, state management — everything you write lives here.



Flutter Framework (Dart)

Material 3 & Cupertino widgets, animation engine, gesture system, painting pipeline.



Flutter Engine (C++)

Skia/Impeller GPU renderer, Dart runtime, platform channels, text layout engine.



Platform Embedder (Kotlin / Swift)

Thin wrapper for each OS — delegates screen, input, and lifecycle events to the engine.

6

Honest Limitations — When NOT to Use Flutter

We're not selling Flutter. We're teaching it. Know the trade-offs.

Platform-Specific APIs

New OS features (iOS Live Activities, Android Dynamic Colour) reach native first. Flutter plugins follow within weeks.

Large Existing Native Team

20 iOS engineers already on payroll? Rewriting in Flutter has negative ROI — it shines for new projects, not migrations.

App Store Review Sensitivity

Apps with many third-party plugins occasionally face extra Apple review scrutiny. Rare and usually resolved.

When Flutter IS the Right Choice

Freelance projects, student capstones, early-stage startups: one codebase, two stores, a fraction of the cost.

7

PART B

Development Environment Setup & Your First Flutter App

Laptops open — everyone types

Target: every student has a running Flutter app before leaving today.

8

Dev Environment Setup — 5 Steps

1

Install Flutter SDK

flutter.dev → Get started → Install → choose OS → extract to ~/flutter (no spaces in path)

2

Add Flutter to PATH

macOS: add `export PATH="$HOME/flutter/bin:$PATH"` to ~/.zshrc then open a new terminal.

3

Install VS Code + Flutter Extension

VS Code → Extensions → search Flutter → Install. Dart extension auto-installs. Restart VS Code.

4

Install Android Studio

Only needed for the Android emulator (AVD). Accept all SDK licences. Create a Pixel 7 device.

5

Run: flutter doctor

Every line must show [✓] before you can build apps. Run in terminal after each fix.

9

flutter doctor — Reading the Output

Run this in your terminal after setup. Every red *X* must be resolved before you can build apps.

Terminal

```
$ flutter doctor
```

```
Doctor summary (to see all details, run flutter doctor -v):
```

```
[✓] Flutter (Channel stable, 3.x.x, on macOS)
```

```
[✓] Android toolchain - develop for Android devices
```

```
[✓] VS Code (version 1.9x)
```

```
[!] Xcode - develop for iOS and macOS
```

```
  X Xcode installation is incomplete.
```

```
[✓] Connected device (Pixel 7 emulator • emulator-5554)
```

Windows students: the Xcode warning above is expected and safe to ignore — you only need the Android toolchain for this course.

10

Create & Run Your First Flutter App

Terminal

```
# Step 1: Create a new project
$ flutter create my_first_app

# Step 2: Open in VS Code
$ cd my_first_app
$ code .

# Step 3: Run the app
$ flutter run

# Or press F5 in VS Code
```

Project Structure

```
my_first_app/
├── lib/
│   └── main.dart
│       └─ YOUR CODE
├── android/ (generated)
├── ios/ (generated)
├── test/
└── pubspec.yaml
    └─ dependencies
```

Hot Reload: save in VS Code or press `r` in terminal — changes appear in under 1 second. Flutter's biggest productivity advantage.